

Guía de laboratorio de la semana 5

Primer proyecto web con Spring Boot

Del portal estático a una aplicación Java ejecutable

Propósito. Preparar un entorno profesional, generar una aplicación Spring Boot reproducible, integrar el frontend del APF1 y demostrar su ejecución mediante pruebas, monitoreo básico y empaquetado.

Resultado de aprendizaje

Al finalizar la semana, el estudiante configura un entorno de desarrollo Java, genera un proyecto Spring Boot con dependencias compatibles, explica su estructura, integra recursos estáticos, ejecuta pruebas y produce un JAR reproducible sin incorporar todavía controladores ni persistencia.

Criterios de salida

- Java 25 es el JDK activo tanto en la terminal como en Visual Studio Code.
- El proyecto fue generado con Spring Boot 4.1.1, Java 25, Maven y empaquetado JAR.
- Las dependencias web usan spring-boot-starter-webmvc; no se fija manualmente la versión de Spring Framework ni Tomcat.
- La clase principal permanece en el paquete raíz `pe.edu.utp.techlab` y la aplicación inicia sin errores.
- El portal de la semana 4 se sirve desde `http://localhost:8080/` y conserva sus recursos y comportamiento.
- El endpoint `/actuator/health` responde con estado HTTP 200 y status UP.
- La prueba de contexto finaliza correctamente y el proyecto produce un JAR ejecutable.
- El repositorio no contiene secretos, archivos generados ni rutas absolutas del equipo.

1 Ficha del laboratorio

Elemento	Descripción
Semana y sesiones	Semana 5, sesiones 9 y 10.
Tema del sílabo	Back end con Java, Spring, Spring Boot y configuración del entorno.
Proyecto acumulativo	Migración del portal TechLab aprobado en el APF1.
Modalidad sugerida	Trabajo individual con contrastación por pares durante las pruebas.
Entregables	Proyecto, README, evidencias, matriz de pruebas y JAR ejecutable.
Conocimientos previos	HTML semántico, CSS, JavaScript, Bootstrap, terminal y estructura de archivos.
Restricción	No añadir base de datos, seguridad, Thymeleaf ni controladores en esta práctica.

1.1 Continuidad del proyecto

Etapas	Responsabilidad	Evidencia
Semana 4	Interfaz Bootstrap ejecutada desde un servidor de desarrollo.	APF1 con HTML, CSS, JavaScript e imágenes.
Semana 5	Spring Boot incorpora proceso Java, servidor embebido, configuración y empaquetado.	Frontend servido por la aplicación y JAR verificable.
Semana 6	Spring MVC atenderá solicitudes mediante controladores y rutas.	Respuestas dinámicas y navegación controlada.

1.2 Recursos necesarios

- Proyecto final de la semana 4 y una copia recuperable antes de migrarlo.
- JDK 25 LTS de una distribución OpenJDK confiable.
- Visual Studio Code con Extension Pack for Java y Spring Boot Extension Pack, o un IDE Java equivalente.
- Conexión a internet durante la primera descarga de Maven y sus dependencias.
- Git recomendado para registrar cambios pequeños y reversibles.

2 Modelo mental de la aplicación

Hasta la semana 4, Live Server entregaba archivos al navegador. Desde esta semana, un proceso Java inicia Spring Boot y un Tomcat embebido escucha solicitudes HTTP. Spring Boot localiza index.html y los recursos del classpath, y devuelve la respuesta al navegador.

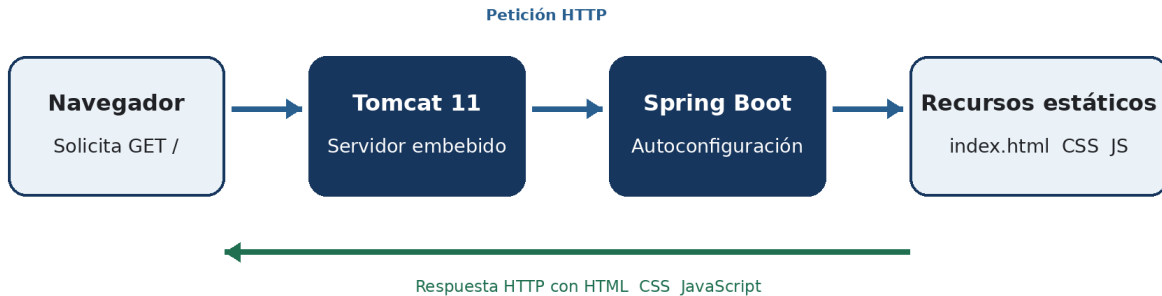


Figura 1 Flujo de una solicitud al portal TechLab

Idea central. El navegador continúa interpretando HTML, CSS y JavaScript. Java no reemplaza el frontend: agrega un proceso de servidor que podrá ejecutar lógica, acceder a datos y generar respuestas dinámicas en las semanas siguientes.

2.1 Responsabilidades por capa

Elemento	Responsabilidad esta semana	Comprobación
Navegador	Solicitar recursos, construir el DOM y ejecutar Bootstrap y JavaScript.	DevTools muestra respuestas 200.
Tomcat embebido	Escuchar el puerto HTTP y transportar solicitudes y respuestas.	El log informa el puerto 8080.
Spring Boot	Crear el contexto, autoconfigurar MVC y administrar componentes.	La aplicación inicia y contextLoads pasa.
Classpath static	Contener index.html, CSS, JavaScript e imágenes públicas.	La raíz / muestra TechLab.
Actuator	Informar el estado básico de ejecución.	/actuator/health devuelve UP.
Maven Wrapper	Resolver dependencias, probar y empaquetar con una versión común.	mvnw test y package finalizan correctamente.

2.2 Spring Framework y Spring Boot

Concepto	Qué aporta	Qué no significa
Spring Framework	Contenedor IoC, inyección de dependencias, Spring MVC y otros módulos.	No es un servidor ni una aplicación terminada.
Spring Boot	Arranque, starters, autoconfiguración, servidor embebido, configuración externa y operación.	No sustituye a Spring Framework; lo configura y organiza.
Spring Initializr	Genera la estructura y el build inicial con combinaciones compatibles.	No desarrolla la solución ni decide su arquitectura.

2.3 Vocabulario esencial

Término	Explicación operativa
---------	-----------------------

Término	Explicación operativa
IoC	El contenedor controla la creación y el ciclo de vida de los objetos administrados.
Bean	Objeto registrado y administrado dentro del ApplicationContext de Spring.
Inyección de dependencias	Un componente recibe por constructor lo que necesita en lugar de crearlo internamente.
Starter	Descriptor de dependencias coherentes para una capacidad, como Spring MVC.
Autoconfiguración	Configuración condicional que Spring Boot aplica según las clases y propiedades disponibles.
Servidor embebido	Servidor incluido en el JAR; no requiere instalar Tomcat por separado.
Classpath	Conjunto de clases y recursos disponibles durante la ejecución de Java.

2.4 Qué concentra SpringBootApplication

Capacidad	Efecto
SpringBootConfiguration	Declara la clase principal como fuente de configuración de Spring Boot.
EnableAutoConfiguration	Activa configuraciones compatibles con las dependencias y propiedades presentes.
ComponentScan	Busca componentes desde el paquete de la clase principal hacia sus subpaquetes.

Buena práctica. Ubique TechLabWebApplication en pe.edu.utp.techlab y todas las clases propias debajo de ese paquete. Evite el paquete por defecto y no coloque la clase principal en un subpaquete lateral.

3 Línea técnica y entorno

La guía fija versiones para que el resultado sea reproducible. Spring Boot administra las versiones compatibles de Spring Framework, Tomcat y dependencias transitivas; por ello, no deben declararse números independientes para esos componentes.

Componente	Línea usada	Criterio
Java	25 LTS	Versión LTS del curso; Spring Boot 4.1.1 admite Java 17 a 26.
Spring Boot	4.1.1	Versión estable verificada al elaborar la guía.
Spring Framework	7.0.9 o superior	Gestionado por Spring Boot; no fijar manualmente.
Spring Web MVC	starter webmvc	Starter vigente; spring-boot-starter-web está obsoleto en 4.1.
Tomcat	11.0.x	Servidor embebido administrado por Spring Boot, Servlet 6.1.
Maven	Wrapper 3.9.16	Generado por Spring Initializr; no exige Maven global.
IDE	Visual Studio Code	Extension Pack for Java y Spring Boot Extension Pack.

3.1 Instalación responsable

1. Instale un JDK 25 completo, no únicamente un JRE. Puede usar Oracle JDK u otra distribución OpenJDK con soporte equivalente.
2. Configure JAVA_HOME para que apunte a la carpeta raíz del JDK y agregue su carpeta bin al PATH.
3. Reinicie la terminal y Visual Studio Code después de cambiar variables del sistema.
4. Instale Extension Pack for Java y Spring Boot Extension Pack desde el Marketplace oficial.
5. No instale Tomcat de forma separada: el starter webmvc incorpora el servidor embebido compatible.
6. No instale Maven global para esta práctica; use siempre el wrapper incluido en el proyecto.

3.2 Verificación previa

Terminal

```
java --version

# macOS o Linux
echo "$JAVA_HOME"

# Windows PowerShell
$env:JAVA_HOME
```

Control	Resultado aprobado	Acción si falla
java --version	Muestra versión 25.	Corregir PATH o seleccionar el JDK activo.
JAVA_HOME	Apunta al JDK 25 sin incluir /bin.	Actualizar la variable y reiniciar terminal e IDE.
Java Projects	VS Code reconoce el runtime 25.	Ejecutar Java Configure Java Runtime.
Extensiones	Ambos paquetes están habilitados.	Instalar o habilitar y recargar la ventana.
Red	start.spring.io y Maven Central son accesibles.	Revisar proxy institucional; no desactivar TLS.

3.3 Puerta de calidad cero

- No continúe mientras la terminal o el IDE muestren otro JDK.
- No copie dependencias de tutoriales antiguos ni cambie versiones internas del BOM de Spring Boot.
- No coloque credenciales, tokens, rutas personales ni información sensible en application.properties.
- Registre una captura legible de la versión de Java y del runtime seleccionado en el IDE.

4 Organización de la semana

Actividad	Salida
Modelo cliente servidor y ecosistema Spring.	Mapa conceptual comprendido.
Instalación y diagnóstico del entorno.	Puerta de calidad cero aprobada.
Generación e inspección del proyecto.	Proyecto importado sin errores.
Primer arranque controlado.	Tomcat escucha en 8080.
Migración del frontend APF1.	Portal servido por Spring Boot.
Componente administrado y configuración.	Log de inicio y propiedades.
Prueba, empaquetado y ejecución del JAR.	Build reproducible.
Evidencias y cierre.	Matriz y README completos.

5 Práctica guiada

Trabaje en incrementos. Cada paso termina con una comprobación antes de modificar el siguiente elemento. Si surge un error, conserve el primer mensaje completo de la consola y diagnostique su causa antes de repetir comandos.

Paso 1 Preparar una copia recuperable

Propósito: proteger el APF1 y separar la versión estática de la nueva aplicación Java.

1. Confirme que el proyecto de la semana 4 abre y supera sus pruebas principales.
2. Cree una copia denominada techlab-apf1-respaldo o etiquete el commit final como apf1-final.
3. Cree una carpeta de trabajo vacía; Spring Initializr generará dentro de ella techlab-web.
4. No mueva todavía los archivos del frontend: primero verifique que el proyecto base inicia.

Evidencia de control. Existe una versión recuperable del APF1 y la carpeta de destino está vacía.

Paso 2 Generar el proyecto con Spring Initializr

Propósito: obtener un proyecto con versiones, wrapper y convenciones compatibles.

1. En VS Code, abra la paleta con Ctrl Shift P y ejecute Spring Initializr Create a Maven Project.
2. Seleccione Spring Boot 4.1.1, Java, Group pe.edu.utp y Artifact techlab-web.
3. Defina Packaging Jar, Java 25 y el paquete pe.edu.utp.techlab.
4. Añada Spring Web, Spring Boot Actuator y Spring Boot DevTools.
5. Seleccione la carpeta de trabajo y abra el proyecto generado cuando VS Code lo solicite.

Evidencia de control. La raíz techlab-web contiene pom.xml, mvnw, mvnw.cmd, .mvn y src.

5.1 Parámetros exactos de Initializr

Campo	Valor
Project	Maven
Language	Java
Spring Boot	4.1.1
Group	pe.edu.utp
Artifact	techlab-web
Name	TechLab Web
Package name	pe.edu.utp.techlab
Packaging	Jar
Java	25
Dependencies	Spring Web, Spring Boot Actuator y Spring Boot DevTools

Resultado en Spring Boot 4.1.1. La opción Spring Web genera spring-boot-starter-webmvc. Si el asistente muestra el nombre amigable Spring Web, no cambie manualmente el starter por una versión antigua.

Paso 3 Importar y esperar la resolución del proyecto

Propósito: permitir que el IDE y Maven construyan el modelo antes de interpretar alertas.

1. Abra exactamente la carpeta que contiene pom.xml, no su carpeta padre.
2. Espere a que Java Projects y Maven terminen de importar; la primera descarga puede tardar varios minutos.

3. Revise la pestaña Problems y diferencie un proceso de descarga activo de un error definitivo.
4. Si aparece una solicitud de confianza del workspace, verifique la ubicación antes de aceptarla.
5. Registre el estado solo cuando desaparezcan los indicadores de carga del proyecto Java.

Evidencia de control. pom.xml es reconocido, la clase principal no tiene errores y Maven muestra el proyecto.

5.2 Estructura inicial

Ruta	Responsabilidad
pom.xml	Modelo Maven, parent de Spring Boot, dependencias y plugin de empaquetado.
mvnw y mvnw.cmd	Comandos del Maven Wrapper para sistemas Unix y Windows.
.mvn/wrapper	Configuración de la distribución Maven fijada por el proyecto.
src/main/java/pe/edu/utp/techlab	Código Java y clase principal en el paquete raíz.
src/main/resources	Configuración, recursos públicos y futuras plantillas.
src/test/java/pe/edu/utp/techlab	Pruebas automatizadas con la misma estructura de paquetes.
target	Salida generada por Maven; no se edita ni se versiona.

5.3 Inspeccionar el POM

No reescriba el archivo generado. Use los fragmentos siguientes como lista de verificación. El parent fija la plataforma compatible y java.version establece el nivel del compilador.

pom.xml Parent y Java

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.1</version>
  <relativePath/>
</parent>

<properties>
  <java.version>25</java.version>
</properties>
```

pom.xml Dependencias de ejecución

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
```

Control de versiones. No agregue version dentro de los starters. El parent de Spring Boot mantiene un conjunto probado; forzar versiones individuales puede producir incompatibilidades difíciles de diagnosticar.

5.4 Dependencias de prueba y build

pom.xml Pruebas y empaquetado

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc-test</artifactId>
  <scope>test</scope>
</dependency>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

- scope test evita que las bibliotecas de prueba formen parte del runtime productivo.
- spring-boot-maven-plugin convierte el resultado en un JAR ejecutable con sus dependencias.
- DevTools es opcional y de runtime; no debe convertirse en requisito para ejecutar el JAR final.

Paso 4 Comprender la clase principal

Propósito: identificar el punto de entrada de Java y la creación del contexto de Spring.

1. Abra TechLabWebApplication.java y confirme que pertenece a pe.edu.utp.techlab.
2. Compruebe que el nombre del archivo coincide exactamente con el de la clase pública.
3. No agregue lógica de negocio al método main; su responsabilidad es iniciar la aplicación.
4. Mantenga las futuras clases en subpaquetes del paquete raíz para que ComponentScan las encuentre.

Evidencia de control. La clase principal coincide con el código de referencia y no muestra errores de compilación.

src/main/java/pe/edu/utp/techlab/TechLabWebApplication.java

```
package pe.edu.utp.techlab;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class TechLabWebApplication {

    public static void main(String[] args) {
        SpringApplication.run(TechLabWebApplication.class, args);
    }
}
```

Elemento	Responsabilidad
main	Punto de entrada estándar de la JVM.
SpringApplication.run	Crea y ejecuta el ApplicationContext de Spring Boot.
SpringBootApplication	Reúne configuración, autoconfiguración y escaneo de componentes.
args	Transporta argumentos externos como --server.port=8081.

Paso 5 Configurar propiedades mínimas

Propósito: declarar valores operativos explícitos sin almacenar secretos.

1. Abra src/main/resources/application.properties.
2. Use un identificador estable y sin espacios para spring.application.name.
3. Mantenga el puerto 8080 como valor por defecto del laboratorio.
4. Exponga únicamente health y conserve ocultos los detalles internos.
5. No agregue usuarios, contraseñas ni tokens a este archivo.

Evidencia de control. Las propiedades son reconocidas por el IDE y no contienen alertas ni datos sensibles.

src/main/resources/application.properties

```
spring.application.name=techlab-web
server.port=8080
management.endpoints.web.exposure.include=health
management.endpoint.health.show-details=never
```

Propiedad	Efecto
spring.application.name	Identifica la aplicación en logs y futuras integraciones.
server.port	Define el puerto HTTP del servidor embebido.
exposure.include	Limita la exposición web de Actuator al endpoint health.
show-details	Evita revelar componentes internos en la respuesta pública.

Paso 6 Realizar el primer arranque

Propósito: aislar cualquier problema del entorno antes de integrar el frontend.

1. Abra una terminal en la raíz que contiene pom.xml.
2. Ejecute el comando del wrapper correspondiente a su sistema.
3. Espere el mensaje de inicio completo y confirme el puerto 8080.
4. Abra /actuator/health y confirme HTTP 200 con status UP.
5. Detenga la aplicación con Ctrl C antes de mover archivos.

Evidencia de control. El proyecto vacío inicia y el endpoint de salud responde correctamente.

Ejecución desde la raíz del proyecto

```
# Windows PowerShell
.\mvnw.cmd spring-boot:run

# macOS o Linux
./mvnw spring-boot:run
```

Primera ejecución. Maven descargará el wrapper y las dependencias si aún no existen en la caché local. No interrumpa el proceso porque una descarga incompleta puede parecer un error del código.

Paso 7 Migrar el frontend del APF1

Propósito: reutilizar la interfaz aprobada sin alterar su estructura ni introducir todavía plantillas dinámicas.

1. Cree src/main/resources/static si el directorio no existe.
2. Copie index.html a static/index.html.
3. Copie las carpetas css, js y assets completas dentro de static.

4. Compruebe que las rutas del HTML siguen siendo relativas y respetan mayúsculas y minúsculas.
5. No use `src/main/webapp` en un proyecto empaquetado como JAR.
6. No coloque el frontend en templates: esa ubicación se reservará para Thymeleaf.

Evidencia de control. La estructura pública conserva todos los archivos y ningún recurso devuelve 404.

5.5 Mapa de migración

Origen semana 4	Destino semana 5
index.html	src/main/resources/static/index.html
css/styles.css	src/main/resources/static/css/styles.css
js/app.js	src/main/resources/static/js/app.js
assets/img	src/main/resources/static/assets/img
README.md	README.md en la raíz del proyecto Java
evidencias	evidencias en la raíz; nunca dentro de static

index.html Rutas relativas desde la raíz pública

```
<link rel="stylesheet" href="css/styles.css">
<script src="js/app.js" defer></script>

```

Comportamiento automático. Spring Boot sirve los recursos de classpath static y usa `index.html` como página de bienvenida. Por eso `http://localhost:8080/` funciona sin escribir un controlador.

Paso 8 Agregar un componente administrado

Propósito: observar IoC, escaneo de componentes, inyección por constructor y logging sin introducir lógica web.

1. Cree el subpaquete `pe.edu.utp.techlab.bootstrap`.
2. Añada `StartupReporter.java` con el código completo de referencia.
3. Observe que la clase recibe `Environment` mediante el constructor y no crea esa dependencia.
4. Inicie la aplicación y localice el mensaje TechLab disponible en el log.
5. No use `System.out.println` para eventos operativos; utilice el sistema de logging disponible.

Evidencia de control. Spring detecta el componente y el log informa nombre y puerto después del arranque.

src/main/java/pe/edu/utp/techlab/bootstrap/StartupReporter.java

```
package pe.edu.utp.techlab.bootstrap;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.core.env.Environment;
import org.springframework.stereotype.Component;

@Component
public final class StartupReporter implements ApplicationRunner {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(StartupReporter.class);

    private final Environment environment;

    public StartupReporter(Environment environment) {
        this.environment = environment;
    }

    @Override
    public void run(ApplicationArguments args) {
        String name = environment.getProperty(
            "spring.application.name", "application");
        String port = environment.getProperty(
            "server.port", "8080");
        LOGGER.info("{} disponible en http://localhost:{}", name, port);
    }
}
```

Elemento	Lectura técnica
Component	Registra StartupReporter como bean detectado por ComponentScan.
ApplicationRunner	Ejecuta run después de crear el contexto de la aplicación.
Constructor	Hace explícita la dependencia Environment y facilita las pruebas.
Environment	Lee propiedades resueltas sin codificar valores operativos en la clase.
Logger	Registra información con nivel y contexto, sin impresión directa a consola.

Paso 9 Verificar el portal desde Spring Boot

Propósito: demostrar que el frontend se entrega mediante el servidor embebido y conserva su calidad.

1. Inicie nuevamente la aplicación con Maven Wrapper.
2. Abra <http://localhost:8080/> y recargue con DevTools Network visible.
3. Confirme que index.html, styles.css, app.js, imágenes y CDN responden sin 404.
4. Repita los casos esenciales del APF1: navbar, modal, formulario, teclado y 360 px.
5. Abra /actuador/health en otra pestaña y confirme la respuesta esperada.
6. No use Live Server durante esta comprobación; la URL debe comenzar con localhost 8080.

Evidencia de control. La interfaz mantiene el comportamiento del APF1 y todas las solicitudes esenciales son correctas.

Comprobación HTTP opcional

```
# macOS o Linux
curl -i http://localhost:8080/actuator/health

# Windows PowerShell
Invoke-WebRequest http://localhost:8080/actuator/health |
    Select-Object StatusCode, Content
```

Resultado esencial esperado

```
HTTP/1.1 200
Content-Type: application/vnd.spring-boot.actuator.v3+json

{"status":"UP"}
```

Paso 10 Ejecutar la prueba de contexto

Propósito: comprobar de forma automatizada que Spring puede construir el ApplicationContext.

1. Abra TechLabWebApplicationTests.java y confirme que usa el mismo paquete raíz.
2. Conserve la prueba contextLoads generada por Initializr.
3. Detenga la aplicación para evitar confundir logs y ejecute el objetivo test.
4. Revise el resumen completo; BUILD SUCCESS y cero fallos constituyen la evidencia válida.
5. Si falla, corrija la primera causa y vuelva a ejecutar toda la suite.

Evidencia de control. Maven informa BUILD SUCCESS y no omite la prueba de contexto.

src/test/java/pe/edu/utp/techlab/TechLabWebApplicationTests.java

```
package pe.edu.utp.techlab;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class TechLabWebApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

Ejecución de pruebas

```
# Windows
.\mvnw.cmd test

# macOS o Linux
./mvnw test
```

Interpretación. contextLoads no sustituye las futuras pruebas funcionales. En esta semana actúa como prueba de humo: detecta errores de configuración, beans imposibles de crear y fallos de arranque del contexto.

Paso 11 Empaquetar y ejecutar el JAR

Propósito: demostrar que el producto funciona fuera del botón Run del IDE.

1. Ejecute clean package desde la raíz; este comando limpia salidas anteriores, compila y prueba.
2. Ubique target/techlab-web-0.0.1-SNAPSHOT.jar y registre su tamaño.
3. Cierre cualquier proceso anterior y ejecute el JAR con java -jar.

4. Verifique nuevamente la raíz y el endpoint de salud.
5. Detenga con Ctrl C y confirme que target permanece excluido por .gitignore.

Evidencia de control. El JAR inicia con Java 25, sirve el portal y responde health sin depender del IDE.

Build y ejecución del artefacto

```
# Windows
.\mvnw.cmd clean package

# macOS o Linux
./mvnw clean package

# Todos los sistemas
java -jar target/techlab-web-0.0.1-SNAPSHOT.jar
```

Definición de terminado. No basta con que la aplicación funcione desde el IDE. La práctica termina cuando las pruebas pasan y el mismo JAR construido inicia correctamente desde una terminal limpia.

6 Reto de aplicación

Implemente un perfil local que cambie el puerto sin modificar la configuración base. Esta actividad demuestra configuración externa y evita mantener valores específicos del equipo en el código.

6.1 Requisitos del reto

1. Cree `src/main/resources/application-local.properties`.
2. Configure únicamente `server.port=8081` en ese archivo.
3. Empaquete el proyecto y active el perfil local mediante un argumento del JAR.
4. Compruebe la raíz y `/actuador/health` en el puerto 8081.
5. Detenga el proceso y ejecute el JAR sin perfil; debe volver al puerto 8080.
6. Documente en README que local es un perfil de desarrollo y no contiene secretos.

`src/main/resources/application-local.properties`

```
server.port=8081
```

macOS o Linux

```
java -jar target/techlab-web-0.0.1-SNAPSHOT.jar \  
+ --spring.profiles.active=local
```

Windows PowerShell

```
java -jar target/techlab-web-0.0.1-SNAPSHOT.jar \  
--spring.profiles.active=local
```

Criterio de aceptación. La configuración base no se edita para cambiar de entorno. El perfil local modifica solo el puerto, y las dos ejecuciones conservan el mismo JAR y el mismo código fuente.

6.2 Revisión del proyecto final

Ruta	Debe contener
<code>src/main/java/pe/edu/utp/techlab</code>	Clase principal y subpaquete bootstrap.
<code>src/main/resources/static</code>	<code>index.html</code> , <code>css</code> , <code>js</code> y <code>assets</code> del APF1.
<code>src/main/resources</code>	<code>application.properties</code> y <code>application-local.properties</code> .
<code>src/test/java/pe/edu/utp/techlab</code>	Prueba <code>contextLoads</code> .
<code>target</code>	JAR construido localmente; excluido de Git.
<code>README.md</code>	Entorno, ejecución, pruebas, perfiles, alcance y evidencias.

7 Protocolo de pruebas

7.1 Casos obligatorios

ID	Procedimiento	Resultado esperado
CP-01	Ejecutar <code>java --version</code> .	JDK activo 25.

ID	Procedimiento	Resultado esperado
CP-02	Ejecutar el proyecto base con Maven Wrapper.	BUILD inicia y Tomcat escucha en 8080.
CP-03	Abrir la raíz en el navegador.	HTTP 200 y portal TechLab visible.
CP-04	Revisar CSS, JS, imágenes y CDN en Network.	Sin 404 ni solicitudes bloqueadas esenciales.
CP-05	Repetir navbar, modal y formulario.	El APF1 conserva su comportamiento.
CP-06	Consultar /actuador/health.	HTTP 200 y status UP sin detalles internos.
CP-07	Revisar el log después del inicio.	StartupReporter informa nombre y puerto.
CP-08	Ejecutar mvnw test.	BUILD SUCCESS y cero fallos.
CP-09	Ejecutar mvnw clean package y luego java -jar.	El JAR inicia y sirve el portal.
CP-10	Activar el perfil local.	El mismo JAR escucha en 8081.
CP-11	Ejecutar sin perfil después del reto.	La aplicación vuelve a 8080.
CP-12	Revisar git status.	No hay target, secretos ni archivos del IDE versionados.

7.2 Registro de resultados

ID	Resultado real	Estado	Corrección aplicada
CP-01		Pasa / Falla	
CP-02		Pasa / Falla	
CP-03		Pasa / Falla	
CP-04		Pasa / Falla	
CP-05		Pasa / Falla	
CP-06		Pasa / Falla	
CP-07		Pasa / Falla	
CP-08		Pasa / Falla	
CP-09		Pasa / Falla	
CP-10		Pasa / Falla	
CP-11		Pasa / Falla	
CP-12		Pasa / Falla	

8 Contenido mínimo del README

Sección	Contenido requerido
Descripción	Propósito del proyecto y continuidad con el APF1.
Requisitos	JDK 25 y conexión inicial para dependencias.

Sección	Contenido requerido
Ejecución	Comandos de wrapper para Windows y sistemas Unix.
Pruebas	Comando test y último resultado.
Empaquetado	Comando package y nombre del JAR.
Perfiles	Puertos base y local; forma de activación.
Alcance	Frontend estático; sin controladores, base de datos ni seguridad.
Evidencias	Ubicación de capturas y matriz de pruebas.
Créditos	Autoría y licencias de imágenes o recursos externos.

8.1 Secuencia de commits sugerida

Mensajes descriptivos

```
chore: generar proyecto Spring Boot 4.1.1
feat: integrar frontend estático del APF1
feat: registrar información de inicio
test: verificar contexto y ejecución del jar
docs: completar README y evidencias
```

- Revise git diff; no versione salidas de build, ajustes del IDE ni secretos.

9 Errores frecuentes y recuperación

Síntoma	Causa probable	Comprobación y solución
invalid target release 25	Maven usa un JDK anterior.	Compare java --version, JAVA_HOME y runtime del IDE; seleccione JDK 25.
JAVA_HOME is not defined correctly	Variable apunta a bin, JRE o ruta inexistente.	Apunte a la raíz del JDK y reinicie terminal e IDE.
mvnw no se reconoce	Terminal fuera de la raíz o comando incorrecto.	Ubíquese junto a pom.xml y use mvnw.cmd en Windows.
Permission denied al ejecutar mvnw	Permiso de ejecución perdido en Unix.	Ejecute chmod +x mvnw y vuelva a probar.
Parent POM no resoluble	Sin red, proxy o descarga incompleta.	Revise conectividad y proxy de Maven; no desactive certificados TLS.
Port 8080 already in use	Otro proceso ocupa el puerto.	Detenga el proceso anterior o use el perfil local en 8081.
La raíz devuelve 404	index.html no está en classpath static.	Mueva el archivo a src/main/resources/static/index.html.
CSS, JS o imágenes devuelven 404	Rutas o mayúsculas no coinciden.	Compare href y src con la estructura real mediante Network.
health devuelve 404	Falta Actuator o se excluyó el endpoint.	Revise el starter y exposure.include=health; reinicie.
StartupReporter no aparece	Clase fuera del paquete escaneado.	Ubíquela debajo de pe.edu.utp.techlab y confirme Component.
contextLoads falla	El contexto no puede crear un componente.	Lea la primera sección Caused by y corrija esa dependencia o propiedad.
El JAR no contiene cambios recientes	No se reconstruyó después de editar.	Detenga, ejecute clean package y vuelva a iniciar el nuevo archivo.

9.1 Estrategia de depuración

1. Reproduzca el fallo con un comando exacto y conserve el mensaje completo.
2. Lea desde la primera línea ERROR y localice la primera sección Caused by.
3. Clasifique el problema como entorno, dependencias, compilación, arranque, HTTP o recurso estático.
4. Aplique una sola corrección y repita el mismo caso de prueba.
5. Después de corregir, ejecute test y una comprobación de regresión del portal.
6. Documente la causa real; no registre como solución únicamente reiniciar o volver a intentar.

11 Preguntas de comprobación

1. ¿Qué responsabilidad añade Spring Boot al portal construido con Bootstrap?
2. ¿Por qué Spring Boot no reemplaza a Spring Framework?
3. ¿Qué tres capacidades concentra SpringBootApplication?
4. ¿Por qué la clase principal debe ubicarse en un paquete raíz?
5. ¿Qué diferencia existe entre un starter y una dependencia aislada?
6. ¿Por qué no debe declararse la versión de Spring Framework dentro del POM?
7. ¿Qué problema evita Maven Wrapper en un equipo de estudiantes?
8. ¿Por qué index.html funciona en la raíz sin un controlador?
9. ¿Qué demuestra contextLoads y qué no demuestra todavía?

10. ¿Qué diferencia existe entre ejecutar `spring-boot:run` y `java -jar`?
11. ¿Por qué se expone únicamente `health` en esta práctica?
12. ¿Cómo demuestra `StartupReporter` la inversión de control y la inyección por constructor?
13. ¿Qué ventaja aporta cambiar el puerto con un perfil en vez de editar el código?
14. ¿Qué partes del proyecto actual serán reemplazadas o extendidas al incorporar Spring MVC?

13 Referencias técnicas oficiales

- [Spring Boot 4.1.1 - Requisitos del sistema](#). Compatibilidad con Java, Maven, Tomcat y Servlet.
- [Spring Boot - Página del proyecto](#). Características, versión estable e inicio rápido.
- [Spring Boot - Sistemas de build y starters](#). Gestión de dependencias y starters vigentes de Spring Boot 4.1.
- [Spring Boot - Estructura del código](#). Paquete raíz, clase principal y escaneo de componentes.
- [Spring Boot - Aplicaciones web Servlet](#). Recursos static, página de bienvenida y Spring MVC.
- [Spring Boot Actuator - Endpoints](#). Endpoint health, exposición y protección de información.
- [Spring Boot - Developer Tools](#). Reinicio y soporte durante el desarrollo.
- [Spring Initializr](#). Generación de proyectos Spring Boot compatibles.
- [Visual Studio Code - Spring Boot](#). Extensiones, creación, ejecución y depuración.
- [Apache Maven Wrapper](#). Ejecución reproducible de Maven por proyecto.
- [Oracle Java SE Support Roadmap](#). Identificación de Java 25 como versión LTS.

Cierre del laboratorio

El portal TechLab ya no es solamente una carpeta abierta por una extensión del editor. Ahora forma parte de una aplicación Java que administra su ciclo de vida, atiende HTTP mediante un servidor embebido, expone un control de salud, ejecuta pruebas y se distribuye como un JAR. Con esta base estable, la siguiente semana podrá incorporar controladores y rutas sin mezclar problemas de entorno con decisiones de arquitectura MVC.